© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

4 Konzepte der Datenmodellierung

4.1 Einleitung

4.1.1 Hintergrund

Im Rahmen der Einführung von Datenbanksystemen in einem Anwendungsumfeld besteht die zentrale Herausforderung weniger in der technischen Auswahl eines leistungsstarken Systems als vielmehr in der Kunst, die für das Anwendungsumfeld notwendigen Daten und ihre Eigenschaften in geeigneter Form als Datenbasis des Datenbanksystems zu konzipieren. Von der inhaltlichen Qualität und Semantik dieser Datenbasis hängt die Nutzbarkeit des Datenbanksystems ab.

Für die systematische Abbildung der mit einem Anwendungsumfeld verbundenen Daten benötigt man entsprechende Methoden und Beschreibungsformen, die den Analyse- und Entwurfsprozess unterstützen. Der Entwurfsprozess wird sich einerseits mit der Frage beschäftigen, wie die fachliche Sicht auf die Daten möglichst ausdrucksstark beschrieben werden kann (**konzeptionelles Datenmodell**) und andererseits wie die systemtechnische Seite des eingesetzten Datenbanksystems entsprechende Datenstrukturen für die Implementierung zur Verfügung stellt (**logisches und physisches Datenmodell**).

Der Ausgangspunkt für die Modellentwicklung ist das Anwendungsumfeld (oftmals als »reale Welt« oder »Problemwelt« bezeichnet) mit all seinen vielfältigen Details und komplexen Zusammenhängen auf der Ebene der relevanten Daten. Auf der Basis eines Abstraktionsprozesses ist dieses Anwendungsumfeld in eine wohldefinierte Beschreibungsform abzubilden, die für die technische Implementierung und auch für die anwendungsseitige Nutzung geeignet erscheint. Die Datenmodellierung fokussiert deshalb auf die Bildung von Datenstrukturen, welche in geeigneter Form genau diejenigen Aspekte der realen Welt beschreiben, die für die Anwendung von Interesse sind. Ein Datenmodell, welches eine hohe Ausdrucksfähigkeit zur Abbildung der realen Sachverhalte hat, nennt sich **semantisches Datenmodell**. Der Einsatz von semantischen Datenmodellen ist deshalb in der fachlichen, konzeptionellen Phase des gesamten Datenbankentwurfs von großer Bedeutung.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

4.1.2 Anforderungen an die Datenmodellierung

Die Datenmodellierung ist subjektiv, d. h. der gleiche reale Sachverhalt kann von verschiedenen modellbildenden Instanzen unterschiedlich abgebildet werden. Die Eigenschaften eines Datenmodelles hängen oftmals von der zugrundeliegenden Zielsetzung ab und können durchaus konkurrierend sein. Im Entwurf von Datenmodellen im Kontext von **operativen Informationssystemen** ist die Vermeidung der Datenredundanz sowie von Speicheranomalien eine wichtige Zielsetzung. Hier steht die Modellierung von **normalisierten Datenstrukturen** im Vordergrund. Normalisierungsregeln beschreiben strukturelle Eigenschaften, die auf die Elimination von Datenredundanz zielen (▶ Kap. 3.7).

Bei der Modellierung von Daten im Umfeld von **analytischen Informationssystemen**, also beispielsweise bei Data Warehouse-Systemen, steht die Analysefähigkeit im Vordergrund. Dementsprechend sind Eigenschaften wie Redundanzfreiheit nicht priorisiert, um die **Performance im Datenzugriff** optimieren zu können. Die Modellierung von **multidimensionalen** und **subjektorientierten Datenstrukturen** steht in diesem Umfeld im Vordergrund. Die Datenstrukturen müssen also eine hohe Analysefähigkeit eröffnen.

Allen Datenmodellen gemein sind Eigenschaften wie Konsistenz, Vollständigkeit und Widerspruchsfreiheit in den modellierten Strukturen.

Für die Prüfung der Erfüllung erwünschter Eigenschaften in den Datenmodellen werden oftmals formale Modellierungsansätze angestrebt. Formale oder zu mindestens semi-formale Modelle (Beispiel: Entity-Relationship-Modell) erlauben oftmals eine fundierte und automatisierte Prüfung von Eigenschaften. Allerdings ist die Akzeptanz von rein formalen Modellen in der Unternehmenspraxis nicht besonders breit verankert.

4.1.3 Logisches und physisches Datenbankdesign

Die Hauptschritte beim Entwurf einer Datenbank lassen sich gemäß dem beschriebenen Phasenmodell in ein **logisches** und ein **physisches Datenbankdesign** unterteilen. Der Hintergrund dieser Grobaufteilung liegt in der unterschiedlichen Zielsetzung sowie der Zuständigkeit im Unternehmen, die jeweils mit den einzelnen Schritten verbunden sind.

Zum **logischen Datenbankdesign** gehören die Schritte zur Erarbeitung der konzeptionellen und der externen Ebene. Hier stehen die Akquisition und das logische Aufteilen der interessierenden Datenobjekte, ihre Eigenschaften und gegenseitigen Beziehungen im Vordergrund. Maßgebliche **Zielsetzung** hierin ist die Erhaltung der Konsistenz und der Integrität der zu verwaltenden Daten.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

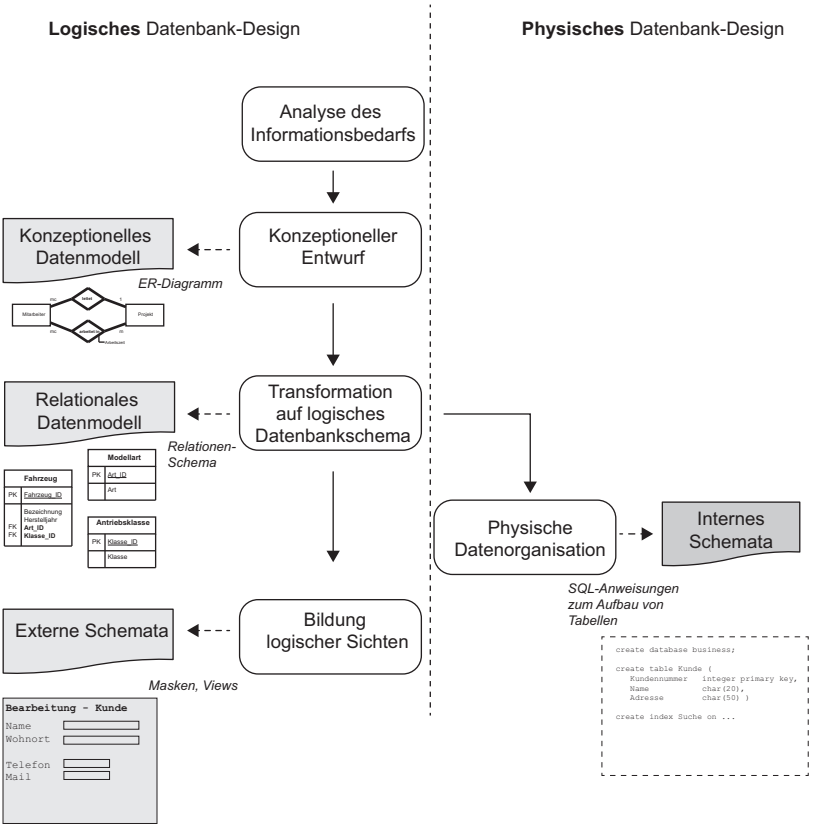


Abb. 4.1: Schritte im Datenbankentwurf

Im **physischen Datenbankdesign** geht es um die Festlegung der internen Ebene. Neben der Auswahl eines Datenbanksystems erfolgt hier die Festlegung der internen Datenorganisation wie beispielsweise Zugriffspfade, die Aufteilung der internen Datenobjekte auf Speichermedien. Das **Ziel** im Entwurfsprozess liegt in der Erreichung einer akzeptablen Performance und der Einhaltung von Anforderungen an die Sicherheit und den Schutz der Daten vor Verlust und unberechtigtem Zugriff.

4.2 Aufgaben im konzeptionellen Entwurf

4.2.1 Analyse des Informationsbedarfs

Im ersten Schritt muss eine Analyse stattfinden, die die relevanten Daten und Informationen im jeweiligen Anwendungsbereich bzw. Unterneh-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

mensbereich identifiziert. Hier gilt es zu hinterfragen, welche Daten und Informationen von Bedeutung sind und für welchen Anwendungszweck sie gebraucht werden.

Beispiel:
Eine Stücklistenstruktur ist für die Mengenplanung und Bedarfsermittlung im Rahmen der Herstellung von Bedeutung.

In der Festlegung der für einen Anwendungsbereich relevanten Daten bietet es sich an, anhand von **Leitfragen** vorzugehen:

- Wer liefert die Daten und Informationen?
  Die Frage nach der Informationsquelle soll sicherstellen, dass die gewünschten Daten im Anwendungsumfeld überhaupt verfügbar sind bzw. bereitgestellt werden können.
- Von wem und wofür werden die Daten benötigt?
  Die Frage nach dem Adressat der Daten und Informationen soll sicherstellen, dass sie überhaupt benötigt werden und im Anwendungsumfeld eine Verwendung finden.

Methodisch kann der Analyseschritt durch Interviewtechniken, Fragebögen und einer Analyse vorhandener Dokumente und Anwendungssysteme umgesetzt werden.

4.2.2 Entwicklung des konzeptionellen Schemas

Das **konzeptionelle Schema** soll die fachliche Sicht auf die Anwendungsdaten beschreiben, frei von technischen Details des späteren Datenbanksystems. Aufgrund des gesetzten Ziels, ein möglichst realitätsgetreues Abbild zu erhalten, werden **semantische Datenmodelle** bevorzugt. In der Unternehmenspraxis hat sich als methodische Grundlage der **Entity-Relationship-Ansatz** nach Chen (1976) bewährt.

Der ER-Modellierungsansatz ist ein praxiserprobter, methodischer Ansatz für die Erstellung von konzeptionellen Modellen zur schematischen Beschreibung der logischen Gesamtsicht auf die Unternehmensdaten. Dieser Ansatz ist ein semi-formaler Modellierungsansatz und bietet eine ansprechende Symbolik für eine Visualisierung des Datenmodells. Die grafische Repräsentation des ER-Modells wird als **ER-Diagramm** bezeichnet.

Der ursprüngliche ER-Ansatz nach Chen wurde hinsichtlich einer stärkeren Ausdrucksfähigkeit systematisch um weitere Modellierungselemen-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

te ergänzt. Hier wird dann oft von einem **Erweiterten ER-Modell**, kurz EER-Modell, gesprochen.

In der Praxis hat der ER-Ansatz nicht zuletzt aufgrund seiner hohen Semantik und der Fähigkeit, im Modellierungsprozess eine schrittweise Verfeinerung und Top-down-Vorgehensweise zu unterstützen, eine breite Anwendung gefunden.

4.3 Modellierung mit dem ER-Ansatz

4.3.1 Übersicht der Strukturierungselemente

Die Modellierung nach dem ER-Ansatz greift auf einen wohldefinierten Satz an **Strukturierungselementen** zurück, die die semantische Ausdrucksfähigkeit des Ansatzes wesentlich prägen:

- **Entity-Typen** und **Beziehungstypen**
- Attribute mit Wertebereichen in den Entity- und Beziehungstypen
- Identifikationsschlüssel
- semantische Integritätsbedingungen

Entsprechend der verfügbaren Modellierungsinstrumente gestaltet sich die Umsetzung des Datenbankentwurfs in nachfolgenden Teilschritten:

- Herausbildung von **Entity-Typen und Beziehungstypen** durch Klassifikation der realen Objekte und konkreten Beziehungen zwischen den Objekten.
- Festlegung von **Attributen** bzw. Eigenschaften der Entity- und Beziehungstypen sowie des **Primärschlüssels** für die Entity-Typen.
- Formulierung von **semantischen Integritätsbedingungen**, die zur Konsistenzerhaltung des Modellinhalts dienen sollen. Insbesondere fällt hierunter die Definition von Komplexitätsgraden für die Beziehungstypen und die beteiligten Entity-Typen.
- Erstellung einer **visualisierten Form** des ER-Modells in Ausprägung eines **ER-Diagramms**.
- **Validierung des Modelles**: Prüfung der Übereinstimmung des ER-Modells mit den Anforderungen zur Abbildung der Daten des Anwendungsumfeldes.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

4.3.2 Entity-Typen

Bedeutung

Das zentrale Strukturierungselement im ER-Ansatz sind **Entity-Typen**, die als Gattungsbegriff eine Menge gleichartiger Objekte, sogenannter Entities, umfassen.

In Abbildung 4.2 wird verdeutlicht, wie Entity-Typen als Ergebnis eines Abstraktionsprozesses aufgebaut werden.

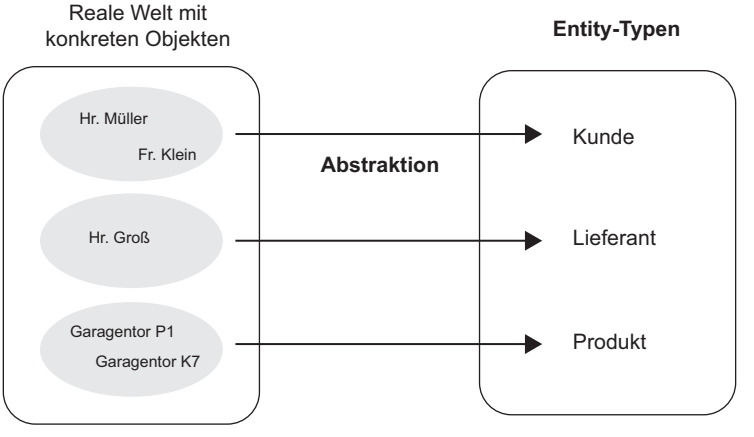


Abb. 4.2: Abstraktionsprozess

In einer ersten Analyse werden konkrete Objekte der Realwelt identifiziert, die für den Anwendungsbereich von Interesse sind, beispielsweise *Hr. Müller*, *Fr. Klein* und *Hr. Groß*.

Entities, die von ihren Eigenschaften als gleichartig angesehen werden, werden einem **Gattungsbegriff** zugeordnet, beispielsweise *Hr. Müller* und *Fr. Klein*, die unsere *Kunden* des Unternehmens sind. Der hieraus gebildete Gattungsbegriff *Kunde* stellt einen **Entity-Typ** dar und verkörpert damit die Klassenbildung der konkreten Entities *Hr. Müller* und *Fr. Klein*.

Attribute von Entity-Typen

Die Eigenschaften der Entity-Typen werden mittels **Attribute** und deren zugehörigen **Wertebereichen** festgelegt. Die Herleitung der Attribute erfolgt aus den notwendigen Eigenschaften zur Charakterisierung der realen Objekte. Die Wertebereiche definieren den möglichen Raum an Werten, die ein Attribut eines Entity-Typs annehmen kann.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

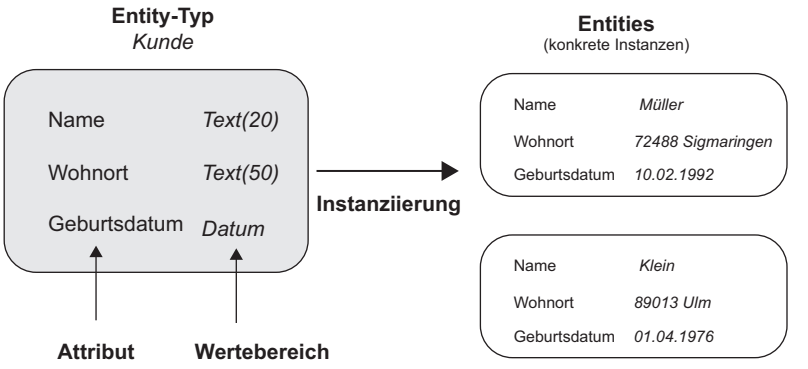


Abb. 4.3: Entity-Typen und Entities am Beispiel

Identifikationsschlüssel von Entity-Typen

In der Strukturierung und Definition eines Entity-Typs mit seinen Attributen ist darauf zu achten, dass die jeweiligen Entities eindeutig identifiziert werden können. Die Eigenschaften bzw. Attribute eines Entity-Typs müssen soweit aufgespannt werden, dass über ein Schlüsselwert zwei Entities immer voneinander unterschieden werden können. Für jeden Entity-Typ wird daher in Analogie zu den Tabellen des relationalen Datenmodells ein **Primärschlüssel** festgelegt.

Ein **Primärschlüssel** ist nach den Ausführungen zum relationalen Datenmodell ein Attribut oder eine minimale Attributkombination, welche zur Identifikation eines jeden Entity dient. Für einen Entity-Typ kann es durchaus mehrere Schlüsselkandidaten geben. Ein ausgewählter Schlüsselkandidat wird zum Primärschlüssel erklärt.

Primärschlüssel müssen folgende Eigenschaften erfüllen:

- Für jedes Entity muss ein Primärschlüsselwert existieren, d. h. der zugehörige Feldwert darf nicht leer sein (sogenannte NULL-Werte sind also nicht erlaubt).
- Der Primärschlüsselwert muss eindeutig sein, d. h. zwei verschiedene Entities dürfen nicht den gleichen Attributwert im Primärschlüssel besitzen.

Zur Vereinfachung wird in der Praxis oft ein **künstlicher Primärschlüssel** eingeführt. Künstliche Primärschlüssel sind meist numerisch und nur aus einem Attribut bestehend.

In der praktischen Modellbildung ist zu beachten, dass die Identifikation von Entities über natürliche Attribute bzw. natürliche Eigenschaften

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

gegeben ist. Zwei Objekte der Realität müssen durch die natürlichen Attribute in der Realität unterscheidbar bleiben. Ein künstlicher Primärschlüssel ist zunächst technisch eine vereinfachte Lösung, darf aber nicht die Identifizierungseigenschaft in der realen Anwendung unterlaufen.

Beispiele:
Für den Entity-Typ *Kunde* sind die Attribute *Name, Geburtstag* und *Wohnort* vorhanden.

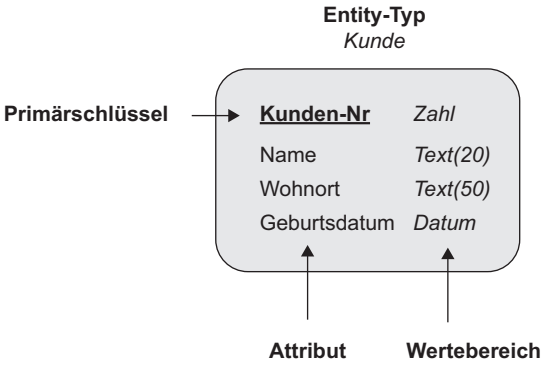


Abb. 4.4: Entity-Typ Kunde mit Primärschlüssel

Die Einführung der *Kunden-Nr* als künstlicher Primärschlüssel ist erst dann sinnvoll, wenn durch die Attribute *Name* und *Wohnort* der Kunde eindeutig bestimmt werden kann. Beispielsweise kann die Postzustellung einer Rechnung nur über die natürlichen Attribute *Name* und *Wohnort*, aber nicht unter Zuhilfenahme über die *Kunden-Nr.* erfolgen.

Notationen und Symbolik für Entity-Typen

Die Beschreibung eines ER-Modelles kann über einen formalen Weg oder über die grafische Darstellung in Form eines ER-Diagramms vorgenommen werden.

Eine **formale Definition** von Entity-Typen könnte wie folgt sein:

```
Entity_Name <   Primärschlüssel     : Wertebereich,
                Attributname        : Wertebereich,
                   … >
```

Beispiel:

```
Kunde <  Kunden-Nr      : Zahl,
         Name           : Text(20),
```

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43


```
        Wohnort        : Text(50),
        Geburtsdatum   : Datum >
```

Für das ER-Diagramm wird zur Darstellung eines Entity-Typs die in Abbildung 4.5 dargestellte Symbolik eingeführt.

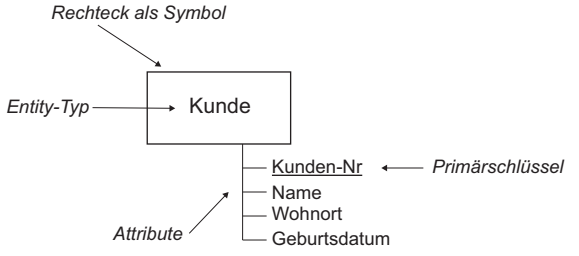


Abb. 4.5: Symbolik für einen Entity-Typ im ER-Diagramm

4.3.3 Beziehungstypen

Bedeutung

In der realen Welt treten **sachlogische Beziehungen** zwischen den Entities auf, die für die Semantik und damit für die Verwendung der Daten von zentraler Bedeutung sind. Im konzeptionellen Datenmodell müssen die sachlogischen Beziehungen über entsprechende Strukturierungselemente abgebildet werden.

Neben dem zuvor eingeführten Strukturierungselement eines Entity-Typs wird der ER-Ansatz um Beziehungstypen erweitert.

Beziehungstypen sind strukturgebende Elemente, die sachlogische Beziehungen zwischen Entities verschiedener Entity-Typen oder auch des gleichen Entity-Typs abstrahieren.

Beispiel:
Gegeben seien die Entity-Typen *Mitarbeiter* und *Projekt* mit jeweiligen Entities. Zwischen den Entities beider Entity-Typen werden folgende konkrete Beziehungen identifiziert:

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

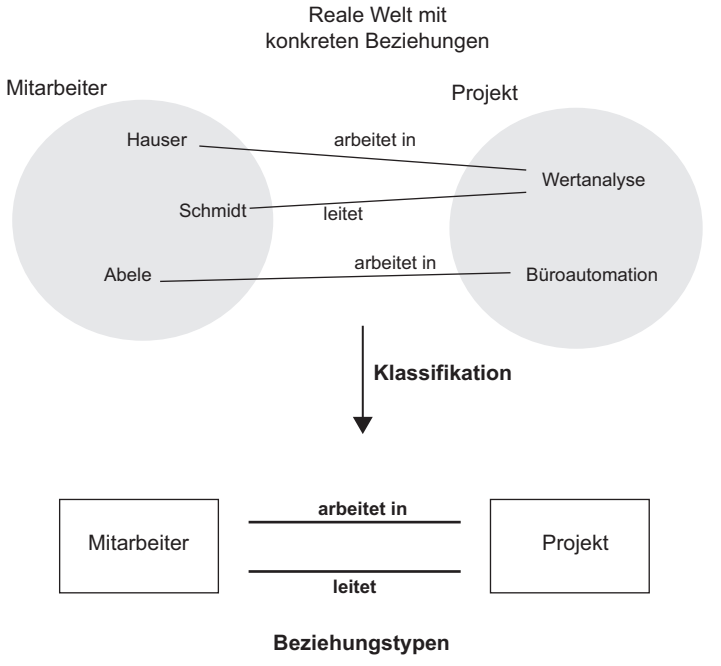


Abb. 4.6: Abstraktion von Beziehungen im ER-Diagramm

Die **Klassifikation** gleichartiger Beziehungen führt zu **Beziehungstypen**, beispielsweise hier zu den Beziehungstypen *arbeitet in* und *leitet*.

Eigenschaften von Beziehungstypen

Bei näherer Betrachtung repräsentiert eine konkrete Beziehung zwischen zwei Entities nichts anderes als ein **Wertepaar** bzw. eine Wertekombination wie beispielsweise (*Hauser, Wertanalyse*). Ein Beziehungstyp selbst beschreibt damit das strukturelle Schema für die Bildung eines Wertepaares zwischen zwei Entities der beteiligten Entity-Typen.

Ein Beziehungstyp ist mit folgenden Eigenschaften versehen:

- Ein Beziehungstyp kann **beschreibende Merkmale** bzw. Attribute besitzen. Beispielsweise kann der Beziehungstyp *arbeitet in* zwischen den Entity-Typen *Projekt* und *Mitarbeiter* das Attribut *Arbeitszeit* enthalten. In diesem Fall spricht man von einem **attributierten Beziehungstyp**.
- Die Beziehungen innerhalb eines Beziehungstyps müssen **identifizierbar**, also unterscheidbar sein. Die gleiche Beziehung darf nicht doppelt aufgenommen werden. Daher muss auch ein Beziehungstyp mit ei-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

nem **Primärschlüssel** ausgestattet sein, der sich in der Regel aus der Primärschlüssel-Kombination der an der Beziehung beteiligten Entity-Typen konstituiert. Bei attributierten Beziehungstypen können im Bedarfsfall auch die beschreibenden Attribute als **Schlüsselattribute** im Primärschlüssel enthalten sein.

- Es sind **referenzielle Integritätsregeln** zu beachten, d. h. eine Beziehung kann nur dann existieren, wenn die im Wertepaar verwendeten Primärschlüsselwerte auch auf existierende Entities in den beteiligten Entity-Typen verweisen.
- Zwischen zwei Entity-Typen können **mehrere Beziehungstypen** mit unterschiedlicher Semantik aufgebaut werden.
- Die **Anzahl** der an einem Beziehungstyp beteiligten Entity-Typen kann beliebig sein und ist damit nicht auf die Zahl zwei beschränkt.

Notation und Symbolik

Die grafische Symbolik zur Darstellung eines Beziehungstyps im ER-Diagramm erfolgt über eine Raute mit **richtungsfreier Verbindung** der beteiligten Entity-Typen (▶ Abb. 4.7).

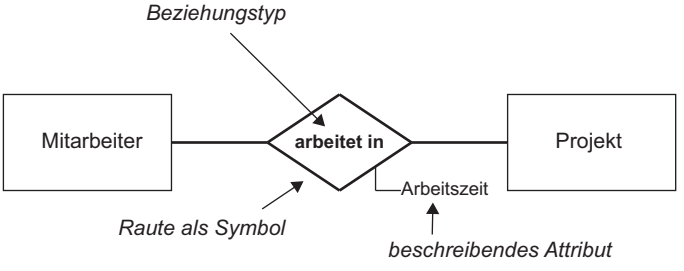


Abb. 4.7: Notation und Symbolik für einen Beziehungstyp

Obwohl in der Interpretation der Beziehung die Leseart hier am Beispiel von links nach rechts mit *Mitarbeiter* **arbeitet in** *Projekt* stattfindet, hat ein Beziehungstyp generell keine Richtung. Die gespiegelte Interpretation des Beziehungstyps aus Sicht des Entity-Typs *Projekt* könnte lauten: *Projekt* **hat** *Mitarbeiter*. Beide Interpretationen sind im gleichen Beziehungstyp *arbeitet in* verkörpert.

Eine **formale Definition** von Beziehungstypen zwischen zwei Entity-Typen E_1 und E_2 könnte wie folgt sein:

```
Beziehungstyp
  < Primärschlüssel_E1      : Wertebereich_E1,
    Primärschlüssel_E2      : Wertebereich_E2,
```

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43


```
    Attributname              : Wertebereich,
        … >
```

Beispiel:
Aus dem Beispiel in Abbildung 4.6 mit den Beziehungen zwischen den beiden Entity-Typen *Mitarbeiter* und *Projekt* resultieren folgende formale Beschreibungen für die beiden Beziehungstypen *leitet* und *arbeitet in*:

```
arbeitet_in < Mitarbeiter-Nr    : Zahl,
              Projekt-Nr        : Zahl,
              Arbeitszeit       : Prozent >

leitet   < Mitarbeiter-Nr      : Zahl,
           Projekt-Nr          : Zahl>
```

Primärschlüssel ist in beiden Beziehungstypen jeweils (*Mitarbeiter-Nr, Projekt-Nr*). Das beschreibende Attribut *Arbeitszeit* ist kein Schlüsselattribut.

Als grafische Darstellung im ER-Diagramm ergibt sich folgendes Bild:

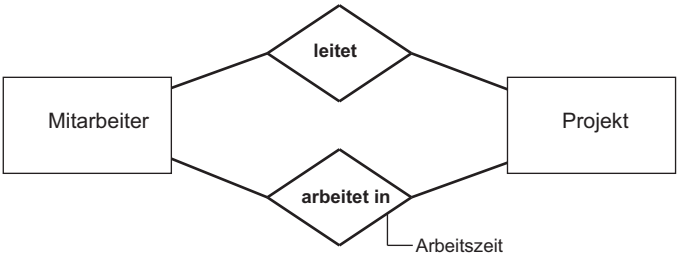


Abb. 4.8: Beziehungstypen am Beispiel

4.3.4 Komplexitätsgrade als Integritätsbedingungen

In der Sachlogik von Beziehungen ist nicht nur relevant, dass Entities miteinander in Beziehung treten, also Kombinationen von Entities als Wertepaare gebildet werden können. Über die Existenz einer Beziehung hinaus, ist für die Sachlogik in der Realität wichtig, die Häufigkeit oder Vielfachheit einer Beziehungsbildung zwischen Entities der beteiligten Entity-Typen genauer zu spezifizieren.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Beispiel:
Zwischen den Entity-Typen *Kunde* und *Rechnung* ist ein Beziehungstyp *gehört* gegeben. Für die Sachlogik in der Rechnungsverwaltung ist es wichtig, die Kombinationsmöglichkeit zwischen *Kunde* und *Rechnung* hinsichtlich der Vielfachheit klar zu spezifizieren: Jede *Rechnung* muss genau einem *Kunden* zugehörig sein; jeder *Kunde* kann beliebig viele *Rechnungen* haben, auch keine.

Die geforderte bzw. erlaubte Vielfachheit in der Bildung von Beziehungen zwischen den Entity-Typen ist eine spezifische Form einer **Integritätsbedingung** an die Daten und wird als Komplexitätsgrad bezeichnet.

Der **Komplexitätsgrad** *n* eines Entity-Typen *E* bzgl. eines Beziehungstyps *F* spezifiziert, an wieviel konkreten Beziehungen jedes Entity *e* aus *E* zu jedem Zeitpunkt beteiligt sein muss bzw. beteiligt sein darf. Formal ausgedrückt gilt:
Kgrad (F,E) = n

In der Definition von Komplexitätsgraden gibt es in der Literatur unterschiedliche Ansätze, die nicht nur in der verwendeten Symbolik, sondern vor allem in der inhaltlichen Ausprägung differenzierte Sichtweisen verfolgen. In diesem Lehrbuch wird eine Definition von Komplexitätsgraden verwendet, die auf den Ansatz nach Schlageter und Stucky (1983) zurückgeht. Wesentliches Argument für diesen Ansatz liegt darin, dass keine Eingrenzung auf zweistellige Beziehungstypen gegeben ist. Die Offenheit in der Anwendung auf **mehrstellige Beziehungstypen** ist in vielen Anwendungsbereichen vorteilhaft, was in der Modellierungspraxis eine **höhere Ausdrucksfähigkeit** impliziert. In der Modellierung von Data Warehouse-Systemen kommt diesem Sachverhalt von mehrstelligen Beziehungstypen eine ganz wesentliche Bedeutung zu.

In der **Notation und Symbolik** von Komplexitätsgraden nach Schlageter/Stucky werden hinsichtlich deren Wertebereichs folgende Möglichkeiten eröffnet:

Symbolik	Kgrad (F,E)	Bedeutung
`1`	`1`	genau 1-mal
`c`	`{0,1}`	optional, also 0- oder 1-mal (kurz: *choice*)
`m`	`[1, ∞)`	mindestens 1-mal (kurz: *multiple*)

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Symbolik	Kgrad (F,E)	Bedeutung
mc	[0, ∞)	beliebig oft (kurz: *multiple choice*)
n	n	genau n-mal
[n_1,n_2]	[n_1,n_2]	zwischen n_1 und n_2 -mal

Die strukturelle Logik eines Datenmodelles, insbesondere bei der Transformation des ER-Modells in das relationale Datenmodell, ist letztlich nur von den vier Komplexitätsgraden *1*, *c*, *mc* und *m* abhängig. Daher begrenzt man sich oft in der Modellierung auf diese vier Symbole. Die semantische Ausdrucksfähigkeit eines Modelles wird natürlich durch die Hinzunahme der Komplexitätsgrade *n* und *[n_1,n_2]* deutlich gestärkt.

Semantik von Komplexitätsgraden

Hinsichtlich der inhaltlichen Bedeutung und dem korrekten Verständnis in der Anwendung der Komplexitätsgrade im ER-Modell ist die Betrachtung der Vielfachheit anhand der konkreten Wertepaare eines Beziehungstyps hilfreich. In Abbildung 4.9 wird die Interpretation verdeutlicht.

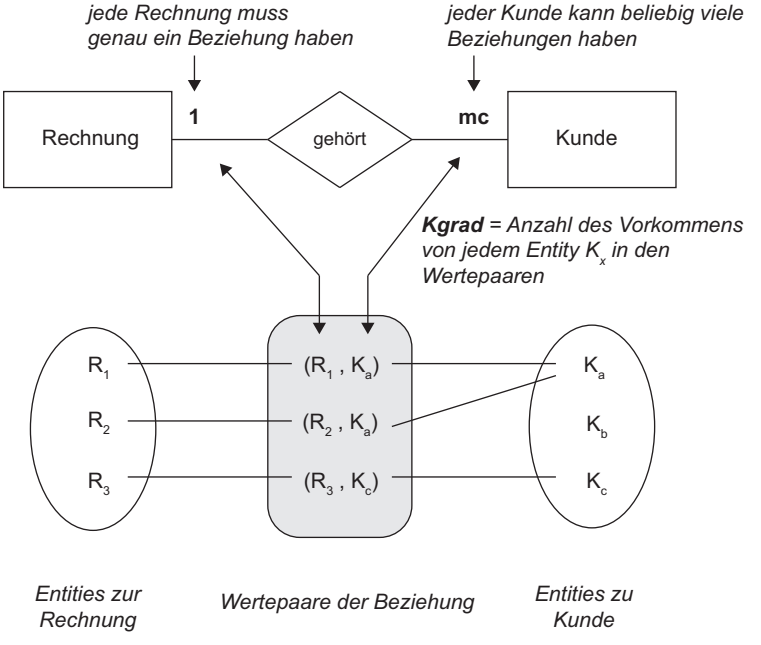


Abb. 4.9: Semantik von Komplexitätsgraden

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Der Komplexitätsgrad K_{grad} *(gehört, Rechnung)* definiert, wie oft jedes Entity aus *Rechnung*, also die Werte R_1, R_2 und R_3 in den Wertepaaren des Beziehungstyps *gehört*, vorkommen muss bzw. darf. Für jede Rechnung R_i muss genau eine Beziehung bzw. ein Wertepaar vorhanden sein und damit muss der Komplexitätsgrad gleich 1 sein, also:

K_{grad} (gehört, Rechnung) = 1

Für den Komplexitätsgrad des Entity-Typs *Kunde* an der Beziehung *gehört* ergibt sich der Komplexitätsgrad *mc*, da jeder Kunde in beliebig vielen Wertepaaren auftreten kann, also:

K_{grad} (gehört, Kunde) = mc

Beispiel:
Die Komplexitätsgrade im einführenden Beispiel mit den Beziehungstypen *arbeitet in* und *leitet* ergeben sich zu:

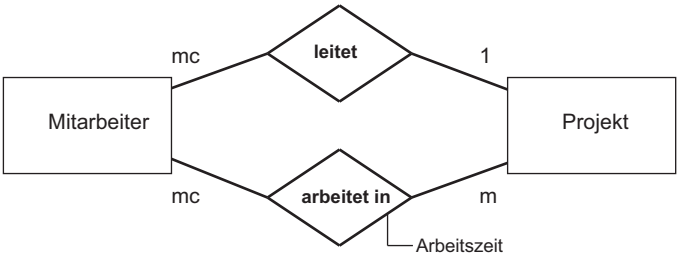


Abb. 4.10: Beispiel für Komplexitätsgrade

Für ein Projekt wird hier angenommen, dass es **genau von einem** Mitarbeiter geleitet wird und **mindestens einen** Mitarbeiter hat.

An diesem Beispiel wird auch deutlich, dass bestimmte Integritätsbedingungen existieren, die im ER-Modell nicht abgebildet werden können. Beispielsweise kann aufgrund der Struktur des Datenmodells ein Mitarbeiter, der das Projekt leitet, gleichzeitig auch als Projektmitarbeiter zugeordnet werden. Ein Mitarbeiter kann in beiden Beziehungen *leitet* und *arbeitet in* enthalten sein. Semantisch korrekt wäre, dass der Projektleiter nicht gleichzeitig als Mitarbeiter geführt werden darf; dies kann im ER-Modell strukturell nicht ausgeschlossen werden.

Die oftmals fehlende semantische Ausdrucksfähigkeit führte zu struktureller Erweiterung des ER-Ansatzes, die im nächsten Kapitel aufgezeigt werden.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

4.4 Erweiterungen zum ER-Modell

Die semantische Ausdrucksfähigkeit des ER-Modells, wie es nach dem Grundansatz von Chen (1976) vorliegt, ist in vielen praktischen Anwendungen der Datenmodellierung nicht ausreichend. Das ER-Modell wurde im Laufe der Zeit durch verschiedenartige Modellerweiterungen in seiner praktischen Anwendbarkeit verbessert. Diese Modellerweiterungen werden im sogenannten **Extended Entity-Relationship-Modell** (kurz: **EER-Modell**) zusammengefasst. Häufig verwendete Ansätze aus dem EER-Modell werden hier im Buch nun vorgestellt und erläutert.

4.4.1 Aggregation

Die **Aggregation** beschreibt die Bildung neuer Objekt-Typen durch die Kombination vorhandener, unterschiedlicher Entity-Typen. In der Problemsituation geht man vom spezifischen Sachverhalt aus, dass das neue Objekt immer die Existenz von Entities der beteiligten Entity-Typen voraussetzt.

Im ER-Modell erfolgt die Realisierung für den neuen Objekt-Typ nicht durch die direkte Erstellung eines neuen Entity-Typen, sondern indirekt durch die Bildung eines **substantiierten Beziehungstyps**. Diese spezifische Art eines Beziehungstyps bringt zum Ausdruck, dass ein konkretes Objekt durch die Aggregation anderer Entities generiert wird.

Beispiel:
Gegeben seien die Entity-Typen *Immobilie* und *Kunde*. Der neue Objekt-Typ *Kaufvertrag* bildet sich aus der Kombination eines *Kunden* und einer *Immobilie*. Ein *Kaufvertrag* hat nur Sinn, wenn ein Kunde und eine Immobilie vorhanden sind.

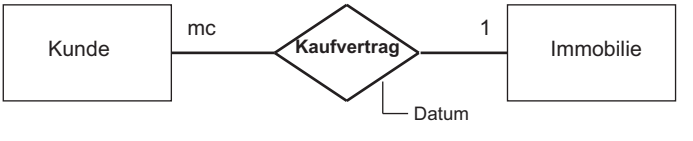


Abb. 4.11: Aggregation als Strukturierungselement

Alternativ kann jede Aggregation auch durch Substitution des Beziehungstyps durch einen Entity-Typ modelliert werden. Daraus ergibt sich folgende Situation:

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

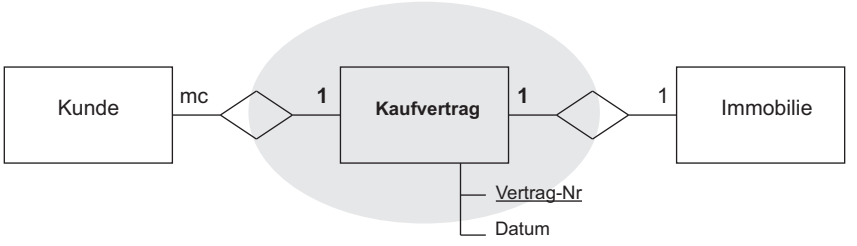


Abb. 4.12: substituierte Aggregation

Die **Komplexitätsgrade** des neuen Entity-Typs *Kaufvertrag* an beiden Beziehungen muss nach der Substitution immer den Wert **1** besitzen; dies ist aufgrund der strukturellen Gegebenheit eines Beziehungstyps zu beachten.

Beide Modelle aus den Abbildungen 4.11 und 4.12 sind inhaltlich in der Datenspeicherung und -verwendung identisch. Der wesentliche Unterschied liegt in der Ausdrucksform und damit in der **semantischen Ausdrucksfähigkeit** durch eine Aggregation.

4.4.2 Uminterpretation von Beziehungstypen

Beziehungstypen sind im ER-Ansatz formal nur zwischen Entity-Typen aufbaubar, nicht jedoch auch zwischen Beziehungstypen bzw. zwischen Entity- und Beziehungstypen. Modelltechnisch kann jedoch ein Beziehungstyp jederzeit auch durch einen Entity-Typ substituiert werden, wie dies bereits in Abbildung 4.12 am Sachverhalt einer Aggregation vorgestellt wurde. Damit können bei entsprechender Vorbereitung beliebige Beziehungen im Modell aufgebaut werden.

Um den Umweg über die Substitution zu ersparen und um letztlich die Interpretation und Lesbarkeit des ER-Modells nicht zu unterlaufen, können Beziehungstypen jederzeit **symbolisch in einen Entity-Typ uminterpretiert** werden. Mit dieser Uminterpretation können damit neue Beziehungstypen auch direkt ausgehend von bestehenden Beziehungstypen aufgebaut werden, wie dies insbesondere bei Aggregationen häufig der Fall ist.

Beispiel:
Ausgehend vom Beispiel der Aggregation *Kaufvertrag* (▶ Abb. 4.11) soll die sachlogische Beziehung eines Kaufvertrags zur notariellen Beur-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

kundung und damit zum betreffenden Notar aufgenommen werden. Jedem Kaufvertrag muss ein Notar zugeordnet werden können.

Die Uminterpretation des Beziehungstyps *Kaufvertrag* erfolgt symbolisch durch Aufsetzen eines Entity-Typs auf *Kaufvertrag*. Im neuen Beziehungstyp *beurkundet* wird *Kaufvertrag* als Entity-Typ gesehen und an dieser Beziehung beteiligt. Dementsprechend erhält der Entity-Typ *Kaufvertrag* auch einen Komplexitätsgrad zugeteilt.

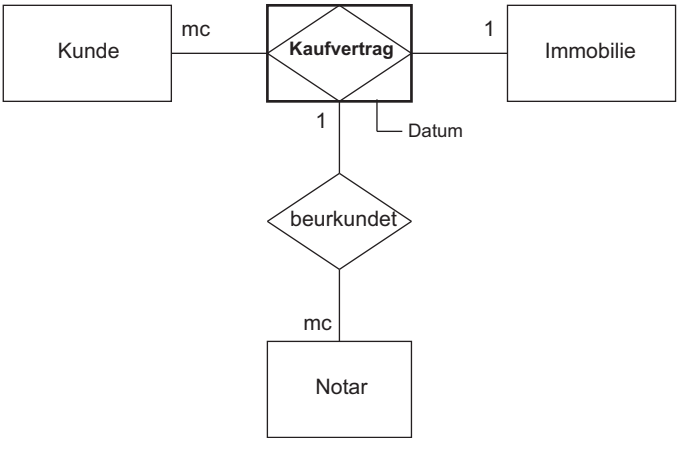


Abb. 4.13: Uminterpretation von Beziehungstypen

4.4.3 Gruppierung

Das Konzept der **Gruppierung** wird eingesetzt, um modellseitig die Einordnung von Objekten in Gruppen beschreiben zu können. Eigentlich handelt es sich hier nicht um eine strukturelle Erweiterung des ER-Modells, sondern um eine spezifische Interpretation eines Beziehungstyps mit bestimmten Komplexitätsgraden.

Zwei Entity-Typen, die durch einen Beziehungstyp mit den Komplexitätsgraden **1** und **m** verbunden sind, repräsentieren eine **hierarchische Struktur** und damit eine Gruppierung. Der beteiligte Entity-Typ mit Komplexitätsgrad *m* repräsentiert die **Gruppen**, der andere Entity-Typ mit Komplexitätsgrad *1* die **Gruppenmitglieder**.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Beispiel:
Häufig werden die Kunden eines Unternehmens in bestimmte Kundengruppen untergliedert, beispielsweise in Privat- und Firmenkunden. Modelltechnisch wird dieser Sachverhalt wie folgt modelliert:

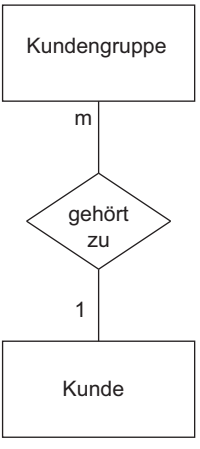


Abb. 4.14: Gruppierung im ER-Modell

4.4.4 Mehrstellige Beziehungstypen

Die bisher betrachteten sachlogischen Zusammenhänge konzentrierten sich auf Beziehungen zwischen **zwei** beteiligten Entity-Typen. In der Realität treten oftmals **mehrere** Entity-Typen miteinander in einen sachlogischen Kontext.

Um die Modellierungssemantik deutlich zu erhöhen, wird die Bildung von Beziehungstypen nicht auf zwei beteiligte Entity-Typen begrenzt, sondern in der Anzahl offengehalten. Der **Grad** eines Beziehungstyps definiert die Zahl der an dem Beziehungstyp beteiligten Entity-Typen. Ist der **Grad größer als zwei**, so spricht man von **mehrstelligen Beziehungstypen**.

Die **Eigenschaften** eines mehrstelligen Beziehungstyps hinsichtlich der Existenz eines Primärschlüssels sowie der Möglichkeit einer Aufnahme beschreibender Attribute, gelten auch analog für mehrstellige Beziehungstypen.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Beispiel:
Als Beispiel für eine mehrstellige Beziehung wird die *mündliche Prüfung* modelliert, an der ein *Student*, ein *Professor* sowie ein *Beisitzer* beteiligt sind.

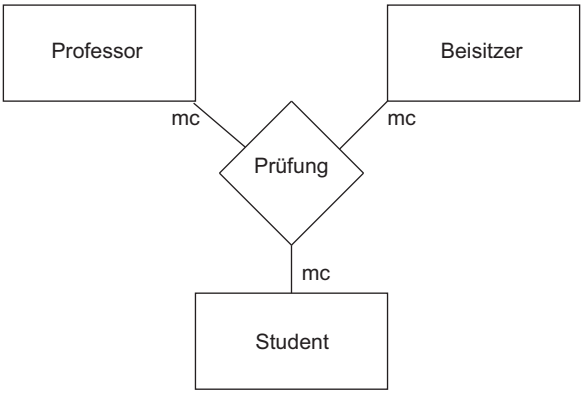


Abb. 4.15: Mehrstellige Beziehungstypen

Die Beziehungen eines mehrstelligen Beziehungstyps stellen nun keine Wertepaare mehr dar, sondern sind analog nun *Werte-Tupel* entsprechend dem Grad *n* des Beziehungstyps, also ein sogenannter *n-Tupel*.

Hinsichtlich der hier im Buch verfolgten Definition von Komplexitätsgraden nach Schlageter/Stucky zeigt sich im Einsatz von mehrstelligen Beziehungstypen ihre Stärke. Die Komplexitätsgrade fokussieren alleine auf die Betrachtung des beteiligten Entity-Typs in seiner Vielfachheit zum Beziehungstyp hin. Die Anzahl weiterer beteiligter Entity-Typen spielt damit keine Rolle, d. h. der Komplexitätsgrad ist in seiner inhaltlichen Bedeutung unabhängig von möglichen, weiteren an diesem Beziehungstyp beteiligten Entity-Typen.

Zu beachten ist, dass ein mehrstelliger Beziehungstyp (*Modell A* ▶ Abb. 4.16) grundsätzlich nicht durch ein Netzwerk von zweistelligen Beziehungstypen zwischen den beteiligten Entity-Typen, (*Modell B*) substituiert werden darf.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

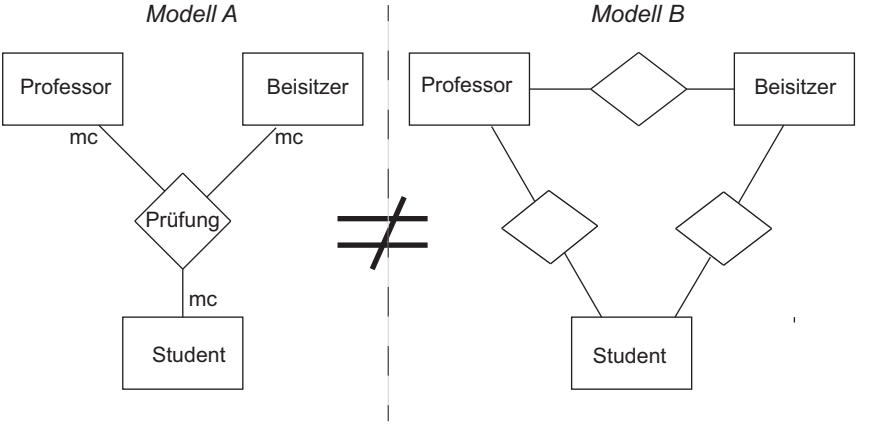


Abb. 4.16: Mehrstellige Beziehungstypen versus zweistellige Beziehungstypen

Zwei ER-Modelle gelten dann als **inhaltsgleich**, wenn die in der Realität auftretenden Daten sowohl in der Möglichkeit der Speicherung als auch in der Wiedergewinnung in beiden Modellen identisch bleiben.

4.4.5 Rollennamen bei Mehrfachbeteiligung von Entity-Typen

In der Modellierung von mehrstelligen Beziehungen können Situationen entstehen, bei denen die Anzahl der beteiligten Entity-Typen nicht dem Grad des mehrstelligen Beziehungstyps entsprechen muss. An folgendem Beispiel soll dies verdeutlicht werden:

Beispiel:
In einer modifizierten Form soll das Beispiel einer mehrstelligen Beziehung aus Abbildung 4.15 betrachtet werden. Die Prüfung, in Form einer schriftlichen Abschlussarbeit (Thesis), wird von einem Erst- und Zweitprüfer abgenommen. Dies führt zunächst zum ER-Modell in Abbildung 4.17.

Die beiden Entity-Typen *Erstprüfer* und *Zweitprüfer* umfassen die gleiche Objektmenge der Professoren unserer Hochschule. Um Redundanz zu vermeiden, müssen die beiden Entity-Typen deshalb zu einem neuen Entity-Typ *Professor* zusammengefasst werden. Hieraus entsteht der Sachverhalt, dass mehrere Entities aus dem identischen Entity-Typ *Professor* am mehrstelligen Beziehungstyp *Thesis* in unterschiedlicher Rolle teilnehmen. Unterschiedliche Rollen der Entities innerhalb einer Beziehung müssen durch separate Verbindungen im Beziehungstyp

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

dargestellt und bezeichnet werden. Deshalb bedarf es der Einführung von **Rollennamen** (▶ Abb. 4.18).

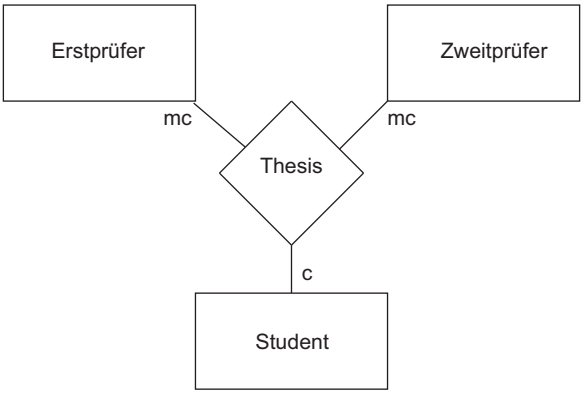


Abb. 4.17: ER-Modell zum Beispiel einer Thesis-Prüfung

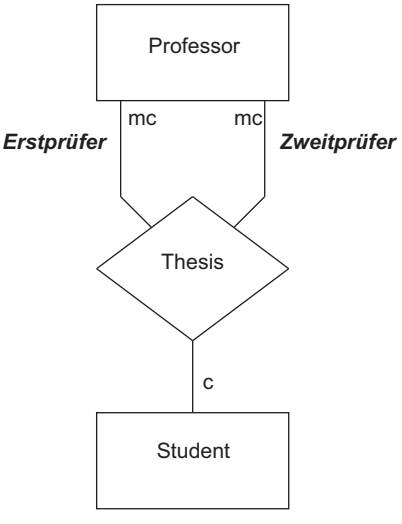


Abb. 4.18: Rollennamen bei einer Mehrfachbeteiligung

In Abbildung 4.18 wird deutlich, dass es sich um einen mehrstelligen Beziehungstyp handelt, an dem aber nur zwei Entity-Typen beteiligt sind. Durch die **Mehrfachbeteiligung** des Entity-Typs *Professor* hat jede konkrete Beziehung genau drei beteiligte Entities. Aufgrund der Rollenvergabe unterscheiden sich die beiden Entities aus dem Entity-Typ *Professor* durch ihre Rollennamen *Erstprüfer* und *Zweitprüfer*. Ein Rol-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

lenname *Kandidat* für das beteiligte Entity aus *Student* könnte ebenfalls eingeführt werden, ist aber nicht zwingend erforderlich.

Die Rollennamen sind bei der Zuordnung von Komplexitätsgraden ebenfalls von Bedeutung. Zu unterschiedlichen Rollennamen können auch unterschiedliche Komplexitätsgrade auftreten. Eine eindeutige Zuordnung ist daher zwingend.

4.4.6 Rekursive Beziehungstypen

Ein Sonderfall der Mehrfachbeteiligung eines Entity-Typs an Beziehungstypen tritt dann auf, wenn die Beziehung ausgehend von einem Entity-Typ auf sich selbst gerichtet ist. Hier spricht man dann von **rekursiven Beziehungen**. Rekursion bedeutet nicht, dass immer nur die gleichen Entities zueinander in Beziehung stehen dürfen, sondern in aller Regel sind es unterschiedliche Entities, die aus dem gleichen Entity-Typ zueinander in Beziehung stehen. Die Rekursion bezieht sich also auf den Entity-Typ und nicht auf die zugehörigen Entities.

In der betrieblichen Praxis treten sehr häufig rekursive Beziehungstypen auf. Beispiele hierfür sind Organigramme und Materialstücklisten. Die Gründe für die Notwendigkeit von rekursiven Modellen sind vielfältig. Neben der semantischen Ausdrucksstärke ist die Vermeidung von Datenredundanz ein weiteres Argument dafür. Dies soll am folgenden Beispiel deutlich gemacht werden.

Beispiel:
Abbildung 4.19 zeigt das Organigramm eines Unternehmens. Hier wird dargestellt, welche Mitarbeiter zu welchen Vorgesetzten gehören.

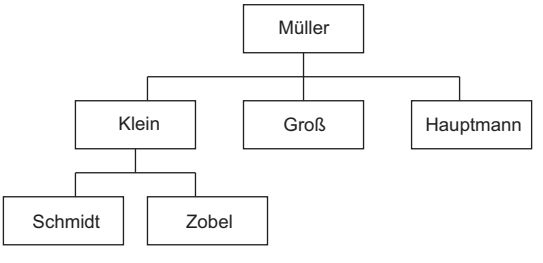


Abb. 4.19: Beispiel eines Organigramms

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Als ersten Entwurf für ein Datenmodell zur Abbildung eines Organigramms werden zwei Entity-Typen *Führungskraft* und *Mitarbeiter* erstellt. Diese beiden Entity-Typen werden über einen Beziehungstyp *ist Chef von* verbunden. Die Komplexitätsgrade ergeben sich aufgrund einer hierarchischen Beziehung mit *1* und *m*.

Folgendes ER-Diagramm entsteht mit beispielhafter Datenbelegung aufgrund des Organigramms aus Abbildung 4.19:

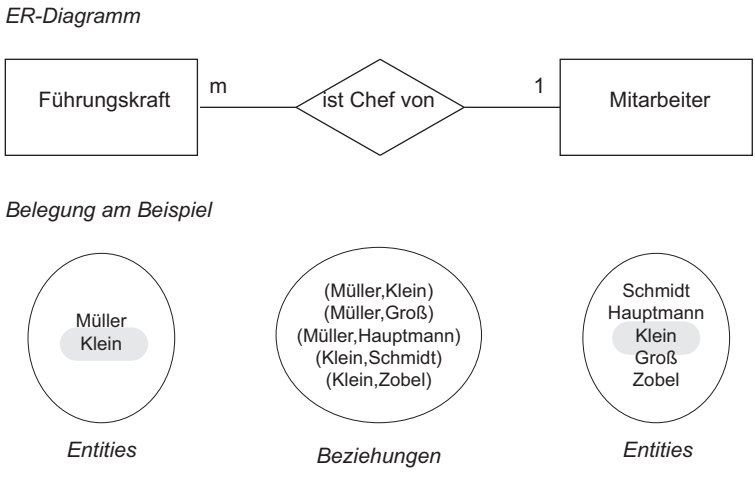


Abb. 4.20: ER-Modell für das Organigramm

Das Datenmodell besitzt den wesentlichen Nachteil, dass in den beiden Entity-Typen die Redundanz an Personendaten erkennbar wird. Alle Personen, die zugleich *Mitarbeiter* wie auch *Führungskraft* sind, also in unterschiedliche Rollen tauchen, sind in beiden Entity-Typen existent. Hier am Beispiel ist dies das Entity *Klein*, welches in beiden Entity-Mengen präsent ist. Sinnvoll erscheint es deshalb, die beiden Entity-Typen *Mitarbeiter* und *Führungskraft* zu einem gemeinsamen Entity-Typ *Person* zu vereinen. Der vorhandene Beziehungstyp geht damit in einen rekursiven Beziehungstyp über. Aufgrund der Mehrfachbeteiligung des Entity-Typs *Person* müssen **Rollennamen** im ER-Diagramm eingeführt werden.

Im Ergebnis ergibt sich das ER-Diagramm mit einem rekursiven Beziehungstyp:

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

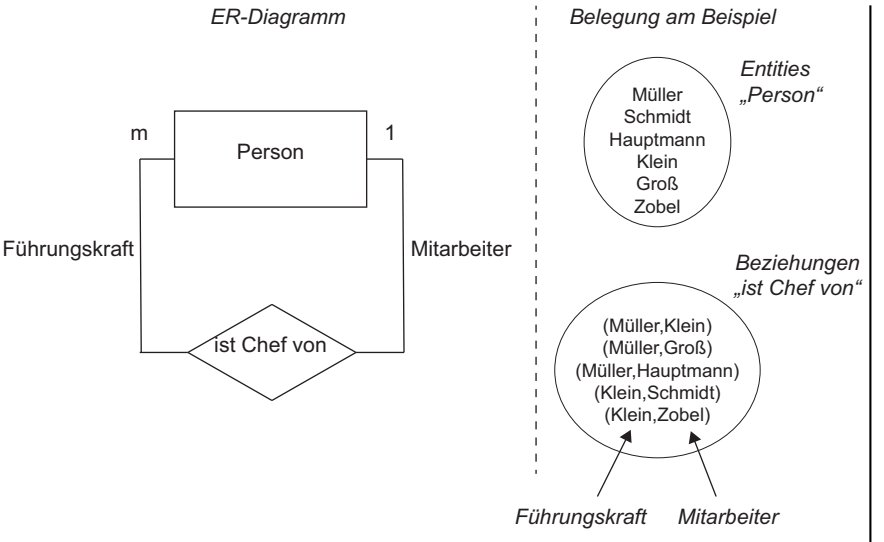


Abb. 4.21: Rekursives ER-Modell für das Organigramm

Hinsichtlich der Komplexitätsgrade muss beachtet werden, dass die ursprünglichen Belegungen mit den Werten *1* und *m* nun mit einem *c* ergänzt werden müssen.

Rekursive Beziehungstypen werden in ihrer Mächtigkeit und ihrer Komplexität oftmals unterschätzt. Obwohl das zugrundliegende Datenmodell denkbar trivial erscheint, können hierin real sehr umfangreiche und sehr dynamische Gebilde gespeichert werden.

4.4.7 Generalisierung und Spezialisierung

In der Entwicklung von ER-Modellen ergeben sich gelegentlich Entity-Typen, die inhaltlich hohe Ähnlichkeit aufweisen. Die Ähnlichkeit besteht darin, dass die Entity-Typen semantisch ähnliche Attributmengen aufweisen und sich oftmals nur durch wenige, spezifische Attribute bzw. Merkmale unterscheiden.

Das Konzept der **Generalisierung** im erweiterten ER-Modell wurde eingeführt, um **inhaltlich ähnliche** Entity-Typen über einen spezifischen Beziehungstyp zu einem **übergeordneten Entity-Typ** zusammenzufassen. Hierzu wird ein besonderer Beziehungstyp, der sogenannte **is-a-Beziehungstyp**, eingesetzt.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Beispiel:
Über die Hochschulorganisation ist nachfolgendes ER-Diagramm vorliegend. Die drei Entity-Typen *Professor, Mitarbeiter* und *Student* sind einer *Fakultät* zugeordnet.

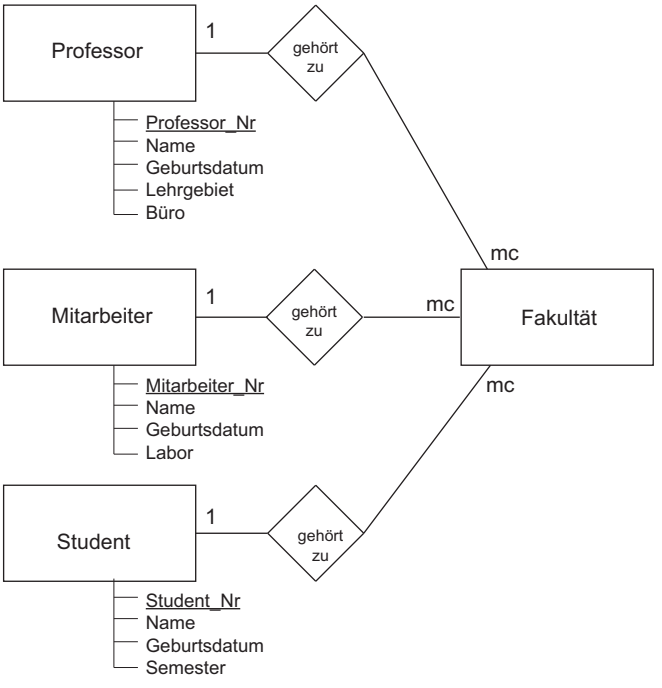


Abb. 4.22: Ausgangssituation für die Generalisierung

Leicht ersichtlich ist, dass die drei Entity-Typen *Professor, Mitarbeiter* und *Student* ähnliche Attributmengen aufweisen. Jeder Entity-Typ verfügt neben dem jeweiligen Primärschlüssel über die Attribute *Name* und *Geburtsdatum*. Darüber hinaus gibt es spezifische Attribute, wie beispielsweise *Lehrgebiet*, welches nur beim Entity-Typ *Professor* vorhanden und auch nur dort sinnvoll ist.

Der Ansatz der Generalisierung besteht nun darin, einen übergeordneten Entity-Typ *Person* einzuführen, welcher die gemeinsam auftretenden Attribute *Name* und *Geburtsdatum* sowie den für alle Entities gültigen Primärschlüssel *Person_Nr* aufnimmt, unter dem die drei zusammengefasst werden könnten. Der neue Entity-Typ *Person* umfasst alle Entities der ausgehenden Entity-Typen *Professor, Mitarbeiter* und *Student*, beschreibt diese aber lediglich anhand der allgemeingültigen

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Attribute *Name* und *Geburtsdatum*. Um die jeweils spezifischen Eigenschaften der ursprünglichen Entity-Typen nicht zu verlieren, werden die ursprünglichen Entity-Typen hinsichtlich dieser Eigenschaften beibehalten.

Das Ergebnis dieser Generalisierung unter Einsatz eines *is-a-Beziehungstyps* führt zu folgendem ER-Modell:

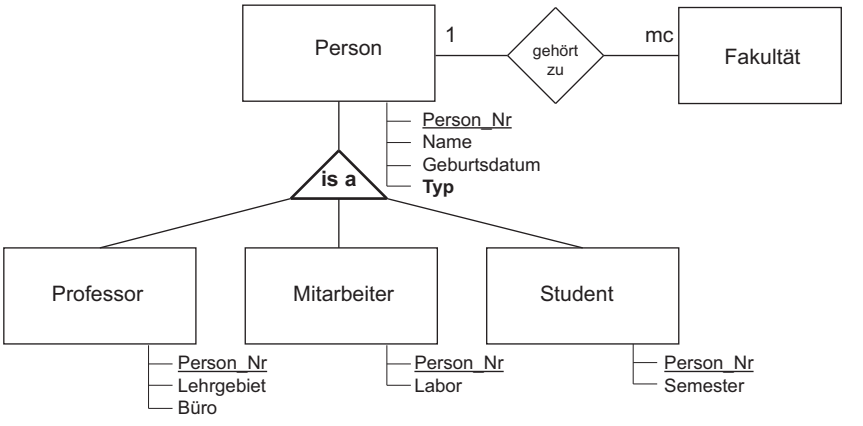


Abb. 4.23: Konzeptionelle Gestaltung einer Generalisierung

Die hier vorgestellte **Bottom-up-Vorgehensweise** abstrahiert aus gegebenen Entity-Typen einen Ober-Typ, der die gemeinsamen Attribute verkörpert. Es erfolgt also eine Generalisierung der spezifischen Entity-Typen mit dem vorgestellten Ergebnis. Die Modellierung einer Generalisierung mithilfe eines *is-a-Beziehungstyps* folgt folgenden Prinzipien und Regeln:

- Der übergreifende bzw. generalisierte Entity-Typ mit den gemeinsamen Attributen wird als **Ober-Typ** bezeichnet. Die ursprünglich gegebenen Entity-Typen werden analog als **Unter-Typen** benannt.
- Es wird ein **gemeinsamer Primärschlüssel** eingeführt, der im Ober-Typ verankert und an die Unter-Typen **vererbt** wird. Dies hat zur Konsequenz, dass jeder existierende Primärschlüsselwert im Ober-Typ in genau einem der Unter-Typen vorhanden sein muss.
- Über den gemeinsamen Primärschlüsselwert werden die für ein Entity gegebenen Attributwerte in ihrer Vollständigkeit aus dem Entity des Ober-Typs und dem korrespondierenden Entity des Unter-Typs zusammengesetzt.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

- Der Ober-Typ erhält alle gemeinsamen Attribute der an der Generalisierung beteiligten Entity-Typen. Darüber hinaus wird noch ein **künstliches Typ-Attribut** aufgenommen, welches die jeweilige Zugehörigkeit eines Entities zum entsprechenden Unter-Typ definiert. Diese Struktur eröffnet eine Klassifizierungsmöglichkeit für alle Entities des Ober-Typs, ohne die Zuordnung zum Unter-Typ über den Primärschlüsselwert vornehmen zu müssen. Für die Optimierung von Datenabfragen ist das Typ-Attribut sehr nützlich.
- Die Generalisierung kann sich auch analog über mehrere Ebenen erstrecken, damit also **mehrstufig** angeordnet sein.
- Dem *is-a-Beziehungstyp* werden **keine Komplexitätsgrade** zugeordnet; aus der besonderen Struktur dieses Elements geht eindeutig hervor, welche Integritätsbedingungen gelten.
- Es können auch Unter-Typen vorkommen, die über den vom Ober-Typ vererbten Primärschlüssel hinaus keine spezifischen Attribute aufweisen. Für die Ausdrucksfähigkeit des Modelles ist es dennoch wichtig, dass diese Unter-Typen geführt werden.

Die Verwendung eines *is-a-Beziehungstyps* kann im Entwurfsprozess der Datenmodellierung unter zwei verschiedenen **Vorgehensweisen** erfolgen:

- Die Bildung eines übergeordneten Entity-Typs aus bereits vorliegenden Entity-Typen mit ähnlichen Attributen wird als **Generalisierung** bezeichnet. Die Generalisierung folgt also einem typischen **Bottom-up**-Ansatz.
- Im Zuge einer schrittweisen Verfeinerung in der Erarbeitung eines ER-Modells kann ein vorliegender abstrakter Entity-Typ systematisch in weitere, spezialisierte Unter-Typen unter Verwendung eines *is-a-Beziehungstyps* unterteilt werden. Hier spricht man dann von einer **Spezialisierung**, die einer **Top-down**-Vorgehensweise im Entwurfsprozess gleichkommt.

Der *is-a-Beziehungstyp* geht von einer überschneidungsfreien Menge von Unter-Typen aus; die Entitätsmengen der Unter-Typen sind also **disjunkt**. Damit kann ein konkretes Entity aus dem Ober-Typ nur genau einem Unter-Typ zugeordnet sein.

Falls die Entitätsmengen der ähnlichen Entity-Typen nicht überschneidungsfrei sind, wird symbolisch der gleiche Beziehungstyp eingesetzt, aber ohne die Bezeichnung *is-a*. In diesem Fall trägt der Ober-Typ auch kein qualifizierendes Typ-Attribut.

In der Abbildung 4.23 mit dem vorliegenden Beispiel der Generalisierung wird deutlich, welche **strukturelle Vorteile** mit diesem Konzept ver-

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

bunden sind. Die ursprünglich vorhandenen, semantisch identischen Beziehungstypen zwischen den Entity-Typen *Professor*, *Mitarbeiter* und *Student* zum Entity-Typ *Fakultät* können nun zu einem einzigen Beziehungstyp zwischen dem Ober-Typ *Person* und dem Entity-Typ *Fakultät* zusammengefasst werden. Damit gewinnt das ER-Modell an Übersichtlichkeit und struktureller Klarheit.

4.5 Transformation in das relationale Datenmodell

4.5.1 Ausgangssituation

Entsprechend den Entwurfsschritten in der Entwicklung eines Datenbanksystems (▶ Abb. 4.1) wird ausgehend von einem fachkonzeptionellen Entwurf eine **schrittweise Transformation** der erarbeiteten Datenmodelle über das logische Datenmodell in die physische Datenorganisation vorgenommen.

Im Einsatz von relationalen Datenbanksystemen muss das vorliegende ER-Modell als Ausprägung des konzeptionellen Datenmodells in ein **relationales Datenmodell** abgebildet werden. Das relationale Datenmodell verkörpert als logisches Datenmodell eine Datenstruktur, die der logischen Datensicht in einem relationalen Datenbankmanagementsystem entspricht. Die Festlegung der physischen Datenorganisation startet dann auf der Grundlage eines relationalen Datenmodells.

Die Erstellung eines relationalen Datenmodells unter Verwendung eines ER-Modells kann nach festen **Transformationsregeln** stattfinden. Es werden hier Regeln definiert, die eindeutig beschreiben, welche Strukturierungselemente im ER-Modell unter welchen Eigenschaften in entsprechende Datenstrukturen des relationalen Datenmodelles abgebildet werden. Dieser Entwurfsschritt lässt sich unter Verwendung dieser Regeln automatisieren, d. h. das ER-Modell lässt sich damit unter Einsatz von Software-Werkzeugen automatisch in ein relationales Datenmodell transformieren.

Die Transformationsregeln erlauben zwar eine automatisierte Abbildung aus dem ER-Modell, dennoch muss beachtet werden, dass das Ergebnis der Transformation, also das logische Datenmodell, eigenen Qualitätsansprüchen genügen muss. Wesentliche **Eigenschaften** des logischen Datenmodells sind insbesondere **Konsistenz** und **Redundanzfreiheit**. Diese Eigenschaften sind im logischen Datenmodell hinsichtlich ihrer Erfüllung zu analysieren und führen ggf. zu weiteren Umgestaltungen in der Datenstruktur.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Damit der Transformationsprozess zu Ergebnissen führt, die hinsichtlich erwünschter Eigenschaften optimal sind, müssen bereits in der konzeptionellen Datenmodellierung die angestrebten Eigenschaften von Konsistenz und Redundanzfreiheit systematisch berücksichtigt werden.

Die Normalisierung von Datenstrukturen spielt bei logischen Datenmodellen eine wichtige Rolle. Der Aufbau eines relationalen Datenmodells (Relationenmodell) hat unter Beachtung verschiedener **Regeln der Normalisierung** (▶ Kap. 3.7) stattzufinden, um redundanzfreie Datenbankstrukturen zu schaffen. Die angestrebte Redundanzfreiheit wird in verschiedenen Stufen durch sogenannte Normalformen systematisch analysiert und aufgebaut. In der konzeptionellen Datenmodellierung sollte deshalb darauf geachtet werden, die Grundsätze der Normalformen zu befolgen, um die Qualität in der nachfolgenden Transformation entscheidend zu erhöhen.

Das weitere Ziel in diesem Abschnitt besteht in der Spezifikation von Abbildungsregeln, die eine adäquate Transformation der Strukturierungselemente des ER-Modelles in ein Relationenmodell erlauben.

Für das Relationenmodell steht als Strukturierungselement nur die Relation bzw. die Tabelle zur Verfügung. Dies bedeutet, dass sämtliche Strukturierungselemente aus dem ER-Modell in Tabellen zu transformieren sind. Zur Abbildung müssen daher Regeln entwickelt werden, wie die Entity-Typen sowie die Beziehungstypen unter Berücksichtigung von Integritätsbedingungen aus dem ER-Modell in entsprechende Tabellen überführt werden. Die nachfolgend beschriebenen Regeln sind an der Zielsetzung einer normalisierten Datenstruktur sowie einer möglichst minimalen Anzahl an notwendigen Relationen orientiert.

4.5.2 Transformation von Entity-Typen

Alle Entity-Typen aus dem ER-Modell werden zunächst 1:1 in jeweilige Tabellen abgebildet. Die vorhandenen Attribute in den Entity-Typen sowie der Primärschlüssel werden als Tabellenspalten übernommen.

Beispiel:
Der Entity-Typ *Lieferant* wird in eine eigene Tabelle *Lieferant* transformiert.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

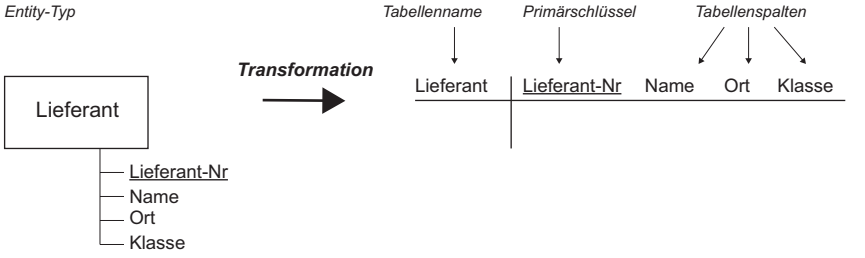


Abb. 4.24: Transformation eines Entity-Typs in eine Relation

In der weiteren Transformation von Beziehungstypen können sich in den bereits erstellten Relationen für die Entity-Typen noch Veränderungen, insbesondere Ergänzungen durch Fremdschlüsselspalten, ergeben.

4.5.3 Transformation von Beziehungstypen

Die Abbildung von Beziehungstypen in das relationale Tabellenschema gestaltet sich etwas komplexer. Beziehungstypen können in eigenständige Relationen transformiert werden oder in bestehende Relationen, durch Aufnahme entsprechender Spalten, Einzug finden. Dies hängt insbesondere von den gegebenen Komplexitätsgraden sowie von möglicherweise beschreibenden Attributen in den Beziehungstypen ab.

Folgende Ausgangssituationen an Beziehungstypen sind für die Aufstellung von Transformationsregeln von Bedeutung:

- Hierarchische sowie konditionale Beziehungstypen
- Netzwerkförmige Beziehungstypen
- Attributierte Beziehungstypen
- Is-a-Beziehungstypen (Generalisierungen)

Hierarchische sowie konditionale Beziehungen

Hierarchische Beziehungstypen liegen vor, wenn mindestens ein am Beziehungstyp beteiligter Entity-Typ einen Komplexitätsgrad vom Wert 1 hat. Liegt ein Wert von *c* (choice) vor, spricht man von einem **konditionalen Beziehungstyp**.

Transformationsregel: Der zu transformierende Beziehungstyp wird durch Aufnahme eines Fremdschlüsselattributs in diejenige Relation

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

abgebildet, die den Entity-Typ repräsentiert, welcher mit dem Komplexitätsgrad vom Wert 1 oder c an dem Beziehungstyp beteiligt ist.

Beispiel:
Gegeben sind die beiden Entity-Typen *Lieferant* und *Rechnung*, die über die hierarchische Beziehung *stammt von* verbunden sind.

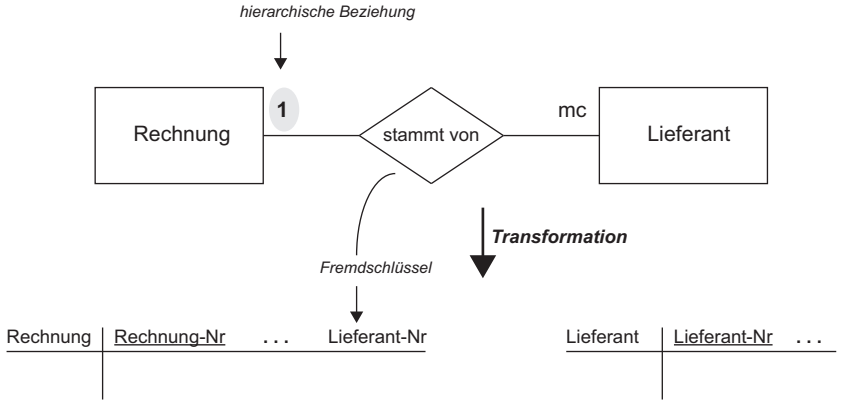


Abb. 4.25: Transformationsregel für hierarchische Beziehungstypen

Bei Vorliegen eines mehrstelligen Beziehungstyps müssen zur Transformation aus allen beteiligten Entity-Typen entsprechend Fremdschlüsselattribute in die Relation aufgenommen werden. Bei einem dreistelligen Beziehungstyp erhält die aufnehmende Relation also insgesamt zwei Fremdschlüsselattribute.

Sind mehrere Entity-Typen am Beziehungstyp mit dem Komplexitätsgrad vom Wert 1 oder c beteiligt, dann kann die aufnehmende Relation frei gewählt werden. Der Beziehungstyp darf aber nur in genau einer Tabelle durch die Fremdschlüsselaufnahme abgebildet werden, nicht mehrfach.

Durchaus möglich ist, dass eine Relation mehrere Fremdschlüsselspalten aufgrund unterschiedlicher Beteiligungen an verschiedenen Beziehungstypen erhält.

Bei der Aufnahme von Fremdschlüsselspalten in die bestehenden Tabellen muss auf eine nachvollziehbare Namensgebung der Spalten geachtet werden. Hinsichtlich einer Eindeutigkeit kann der korrespondierende Tabellenname den Spaltennamen qualifizierend vorangestellt werden.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Netzwerkförmige Beziehungstypen

Netzwerkförmige Beziehungstypen liegen vor, wenn kein vorhandener Komplexitätsgrad eines beteiligten Entity-Typs den Wert 1 oder c aufweist, also alle Komplexitätsgrade **m** oder **mc** sind.

Transformationsregel: Der zu transformierende Beziehungstyp wird durch Erstellung einer neuen, selbstständigen Relation abgebildet. Die neue Relation bildet sich durch die Aufnahme der jeweiligen Primärschlüsselattribute der beteiligten Entity-Typen als Fremdschlüsselattribute in der neuen Relation.

Beispiel:
Gegeben sind die beiden Entity-Typen *Lieferant* und *Artikel*, die über die netzwerkförmige Beziehung *lieferbar* verbunden sind.

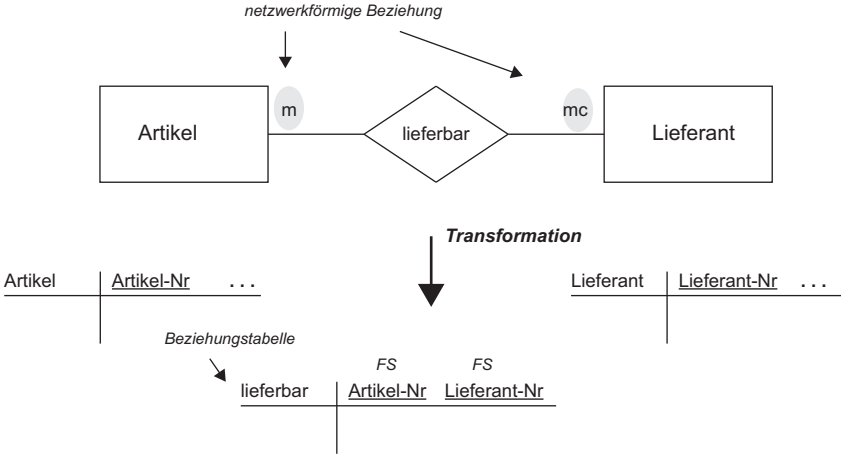


Abb. 4.26: Transformationsregel für netzwerkförmige Beziehungstypen

Die Abbildung des Beziehungstyps über die Aufnahme von Fremdschlüsselattributen in bestehende Relationen der beteiligten Entity-Typen wäre technisch möglich, würde aber dadurch Redundanz in den bestehenden Relationen erzeugen. Aufgrund der Zielsetzung einer **redundanzfreien Datenstruktur** muss die Transformation durch Bildung einer neuen Relation erfolgen. Die neue Relation wird oftmals als Beziehungstabelle bezeichnet, obwohl es im relationalen Datenmodell immer nur gleichwertige Tabellen gibt und für das Datenbanksystem eine solche Unterscheidung nicht wirklich vorhanden ist.

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Im Beispiel der Abbildung 4.26 dient die neue Relation *lieferbar* (Name ist frei wählbar) zur Umsetzung des Beziehungstyps *lieferbar*. Die Relation erhält die beiden Fremdschlüsselattribute *Artikel-Nr* und *Lieferant-Nr*, die zu den Primärschlüsseln der beteiligten Entity-Typen korrespondieren. Beide Fremdschlüsselattribute sind in Kombination zugleich Primärschlüssel der neuen Tabelle *lieferbar*.

Attributierte Beziehungstypen

Besitzen Beziehungstypen beschreibende Attribute, die den einzelnen Beziehungen Eigenschaften zuordnen, so wird die Transformation in das Relationenmodell losgelöst von den vorhandenen Komplexitätsgraden vorgenommen.

Transformationsregel: Ein attributierter Beziehungstyp wird unabhängig der vorhandenen Komplexitätsgrade durch Erstellung einer selbstständigen Relation abgebildet. Die neue Relation bildet sich durch die Aufnahme der jeweiligen Primärschlüsselattribute der beteiligten Entity-Typen als Fremdschlüsselattribute. In der neuen Relation werden die beschreibenden Attribute des Beziehungstyps ebenfalls mit aufgenommen.

Beispiel:
Gegeben sind die beiden Entity-Typen *Lieferant* und *Einkäufer*, die über den Beziehungstyp *zuständig* verbunden sind. Das Attribut *Budget* beschreibt die Maximalgrenze für das Liefervolumen.

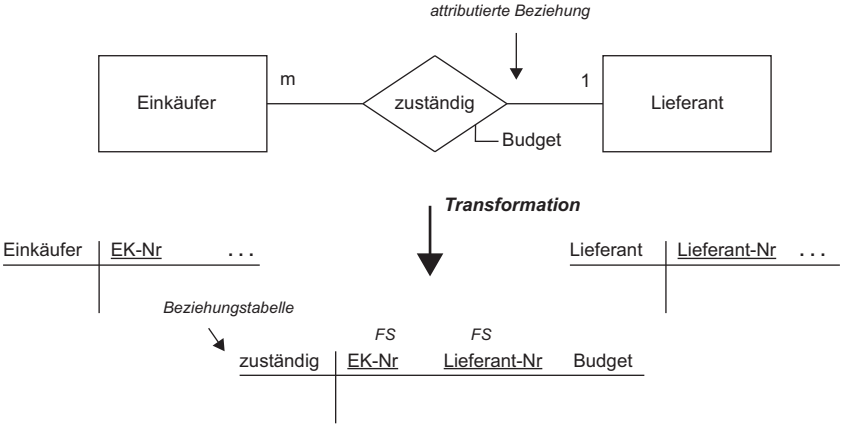


Abb. 4.27: Transformationsregel für attributierte Beziehungstypen

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Unabhängig vom Komplexitätsgrad mit dem Wert 1 wird eine neue Relation aufgebaut, die den Beziehungstyp zuständig verkörpert. Im Hintergrund dieser Transformationsregel steht die Erreichung bestimmter Normalformen in den entstehenden Relationen.

is-a-Beziehungstypen (Generalisierungen)

Generalisierungen, die über ***is-a*-Beziehungstypen** definiert werden, lassen sich ebenfalls nach einem festen Schema transformieren. Die aus den beteiligten Entity-Typ resultierenden Relationen enthalten aufgrund der Vererbung des Primärschlüssels aus dem Ober-Typ an die zugehörigen Untertypen bereits die notwendigen Fremdschlüsselbeziehungen.

Transformationsregel: Der *is-a*-Beziehungstyp einer Generalisierung wird implizit durch die Bildung der Relationen aus den an der Generalisierung beteiligten Entity-Typen abgebildet. Der Primärschlüssel der Relation des Ober-Typs wird an alle Relationen der Unter-Typen vererbt; der Primärschlüssel der Relationen für die Unter-Typen ist zugleich Fremdschlüssel und damit Abbild des *is-a*-Beziehungstyps.

Beispiel:
Gegeben ist die Generalisierung aus Abbildung 4.23. Die Umsetzung in das Relationenmodell resultiert in folgendem Ergebnis:

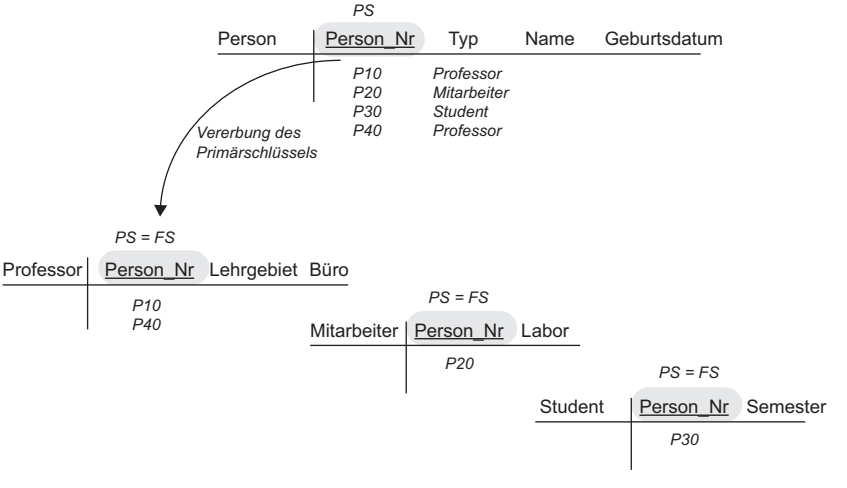


Abb. 4.28: Transformationsregel für Generalisierungen

© Dies ist urheberrechtlich geschütztes Material. Bereitgestellt von: ZHAW Zürich Fr, Feb 14th 2020, 10:43

Die beispielhaften Tupel in den Relationen zeigen auf, wie die Primärschlüsselwerte aus der Relation *Person* auf die Relationen *Professor, Mitarbeiter* und *Student* verteilt werden. Im Attributwert von *Typ*, als Spalte der Relation *Person*, wird die Zuordnung angezeigt. Damit kann bereits in der Relation *Person* die Spezialisierung eines Datensatzes abgelesen und benutzt werden.

Literaturverzeichnis

Chen, Peter Pin-Shan (1976): The entity-relationship model—toward a unified view of data. In: ACM Trans. Database Syst. 1 (1), S. 9–36. DOI: 10.1145/320434.320440.

Schlageter, Gunter; Stucky, Wolffried (1983): Datenbanksysteme: Konzepte und Modelle. 2., neubearbeitete und erweiterte Auflage. Stuttgart: Teubner.